

Analyse des performances mémoire lors du chargement de séries temporelles BSP3

Ce projet étudie les performances en mémoire lors du chargement de données de séries temporelles via des API, dans un scénario de type cloud inspiré d'une entreprise du secteur de l'énergie. Les données sont stockées dans une base de données NoSQL Apache Cassandra exécutée dans un conteneur Docker, et les API sont utilisées pour récupérer et traiter des séries temporelles, principalement des Open Positions (OP) pour 10 books. Le principal problème est que l'API Python présente une augmentation rapide et continue de l'utilisation de la RAM lorsque l'on appelle de manière répétée l'endpoint qui charge les séries temporelles, et que cette mémoire ne redescend pas ensuite. Cela peut finir par atteindre la limite de mémoire de la machine/du conteneur, forçant l'application à redémarrer, ce qui peut provoquer des interruptions et des résultats incohérents.

L'objectif de l'étude est de comparer deux implémentations backend manipulant le même dataset Cassandra : une API en Python 3.12.6 développée avec Flask 3.1.2, et une API en Java 21.0.6 développée avec Spring Boot 2.7.18. La comparaison se concentre sur les performances et l'efficacité mémoire, afin d'aider à faire des choix technologiques plus pertinents lorsque l'utilisation de la mémoire est une contrainte importante. Pour mesurer le problème, des scripts de monitoring et des endpoints supplémentaires ont été créés afin de suivre à la fois la mémoire du processus et la mémoire globale du système pendant l'exécution de plusieurs requêtes. Les expériences ont été réalisées sur un MacBook Pro M1 avec 8 Go de RAM.

Les tests initiaux ont montré que l'application Python consommait environ 24 Mo en moyenne lorsqu'elle était au repos, mais que lors d'appels répétés de l'endpoint problématique, la mémoire augmentait d'environ 85 Mo jusqu'à environ 155 Mo après 10 appels, ce qui montre une forte hausse sans retour au niveau initial. Pour réduire cela, une méthode de nettoyage mémoire plus agressive a été ajoutée en Python, en forçant le garbage collector et en vidant certains caches (y compris des fonctions spécifiques à Linux). Avec ce nettoyage activé, la mémoire a tout de même augmenté rapidement au début à cause du démarrage et du caching, mais la croissance est ensuite devenue plus lente et a fini par se stabiliser, ce qui indique que le comportement mémoire s'est amélioré et ne continuait plus à augmenter indéfiniment.

Pour l'API Java Spring Boot, la mémoire a été suivie à l'aide du RSS et du Heap. Sans appel à l'endpoint principal, elle utilisait environ 20 Mo de heap. Après des appels répétés de l'endpoint, les deux valeurs ont augmenté ; le heap a brièvement diminué grâce au garbage collector, mais il est ensuite remonté et n'est pas revenu au niveau initial, tandis que le RSS est redescendu à un niveau proche de celui du départ.

Globalement, le rapport conclut que Python et Java réalisent la même tâche fonctionnelle, mais suivent des styles de développement différents : Python/Flask est plus court, flexible et adapté à l'expérimentation ainsi qu'à la manipulation de séries temporelles (notamment avec Pandas), tandis que Java/Spring est plus structuré, plus prévisible et plus adapté aux systèmes de production de longue durée, où la stabilité et le contrôle des ressources sont essentiels.

